



FAMRES: A Framework for Automated Testing of Microservices Based on Requirement Specifications

Hoang-Viet Tran^(✉), Nguyen Phuong Chi, and Pham Ngoc Hung

University of Engineering and Technology, Vietnam National University, No. 144
Xuan Thuy Street, Dich Vong Ward, Cau Giay District, Hanoi, Vietnam
{thv, 23025003, hungpn}@vnu.edu.vn

Abstract. Quality assurance for microservices has become increasingly important in software engineering as more distributed systems adopt the microservices architecture. This paper introduces FAMRES, a microservices testing framework based on requirement specifications. Initially, we present an XML-based structure for these specifications. FAMRES uses the predefined requirements to generate the necessary test data. Based on the generated test data, software testers must calculate the corresponding expected outputs based on the predefined requirements. The test data is subsequently used to invoke the microservice APIs developed from those requirements. Finally, the framework generates test reports based on the API invocation results. We have conducted initial experiments with some commonly used APIs in the research community, showing promising results.

Keywords: Automated testing · microservices · test data generation · requirement specification

1 Introduction

Microservices architecture organizes an application into a collection of loosely coupled, fine-grained services [1, 2]. These services interact through lightweight protocols such as REST API calls [3] and Google Remote Procedure Calls (gRPC)¹. This approach allows development teams to develop and deploy services independently, thereby reducing code interdependency and enhancing readability and modularity within the project.

Due to its widespread popularity, ensuring quality assurance for microservices has emerged as a critical challenge in software development. This issue has prompted the publication of numerous research papers addressing various aspects of the problem. Thomas et al. provide an overview of quality assurance for microservices architecture, identifying various architectural smells and anti-patterns [4]. Bogner et al. explore how practitioners manage evolvability assurance processes for microservices, focusing on the tools, metrics, patterns utilized,

¹ <https://grpc.io/>.

and challenges faced in maintaining their systems' evolvability [5]. Venkat et al. investigate automated testing strategies for microservices within a DevOps framework, offering insights into best practices, tools, and methodologies [6]. Giamattei et al. introduce a grey-box strategy named MACROHIVE for automated testing and monitoring of microservices interactions. MACROHIVE uses combinatorial testing to generate valid and invalid tests from microservices specifications in the OpenAPI/Swagger format [7]. Numerous other papers have also addressed related issues in microservices [8–11]. However, to our knowledge, no paper has proposed a method for automatically testing microservices from requirement specification through to the generation of test reports.

This paper introduces a framework named FAMRES designed for testing microservices with requirements specified in XML files. Each XML file details the parameters of an API provided by the microservices. The framework comprises four main phases. First, FAMRES automatically generates test data for the API based on the requirement specification using Boundary Value Analysis (BVA) and Equivalence Class Partitioning (ECP) methods. Second, corresponding test cases are manually created using the generated test data and the requirement specification. Each test case includes input test data and the expected output. Third, The development team implements the microservices API, producing the corresponding source code. This source code is then deployed to a server as microservices. In the final phase, the framework automatically invokes the microservice API, retrieves the response, compares it to the expected output from Phase 2, and generates the necessary reports. We have implemented FAMRES as a tool and conducted some initial experiments, yielding promising results.

The rest of this paper is organized as follows: Sect. 2 provides an overview of the FAMRES framework. Then, Sect. 3 details the method for describing the requirement specification of the microservice API. In Sect. 4, we explain how test data is automatically generated from the requirement specification. Section 5 outlines the method for invoking the microservice API for test execution. Preliminary experiments are discussed in Sect. 6. Finally, we conclude the paper in Sect. 7.

2 An Overview of FAMRES

Given a software requirement in the form of XML files, the FAMRES framework automates the processes of test data generation, microservice API test execution, and test report generation. Figure 1 provides an overview of the framework, which comprises the following four phases:

- Phase 1 (Test Data Generation): In this phase, the framework utilizes the BVA and ECP methods to automatically generate the necessary test data. The test data generation process accepts the requirements XML files, each containing pairs (p_i, V_i) , where p_i represents a parameter of the microservice API and V_i is a sorted list of values $\{v_1, v_2, \dots, v_n\}$ with $v_i \leq v_{i+1}$,

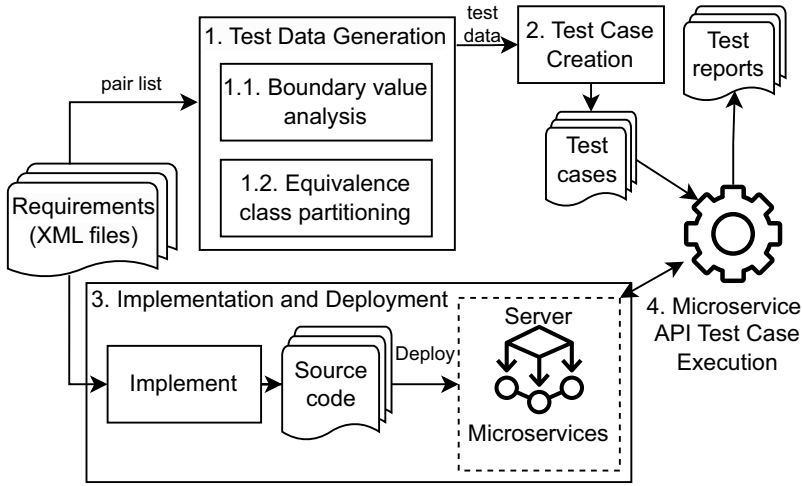


Fig. 1. An overview of FAMRES framework.

$\forall 1 \leq i \leq n - 1$. In this phase, we have employed the BVA and ECP methods within FAMRES. However, other black-box testing methods, such as pairwise testing [12], state transition testing [13], decision table testing [14], etc., can also be used to generate test data from the requirements.

- Phase 2 (Test Case Creation): In this phase, based on the requirements of the microservices, testers need to calculate the corresponding output for each test data generated in Phase 1. As far as we know, this cannot be done automatically, so it must be performed manually.
- Phase 3 (Implementation and Deployment): In this phase, developers implement the requirements to create the necessary microservices. The resulting source code is then deployed to a server to run and provide the required API. This API will be invoked with the test cases generated in Phase 2 to validate the correctness of the microservice implementation.
- Phase 4 (Microservice API Test Case Execution): The microservice API is automatically invoked using the test cases generated in Phase 2. Upon execution of all test cases, the framework generates the necessary test report, detailing any errors found in the corresponding test cases.

3 Requirement Specification for Automated Test Data Generation

This framework represents requirement specifications using XML files with a predefined format. This allows the framework to read information about each input parameter. Listing 1 provides an example of a requirement for an API that calculates the fare based on two parameters: age and distance traveled. In

Listing 1, please note that “>” should be “>”, “<” should be “<”, and “&” should be “&” within the contents of XML tags. We keep them as “>”, “<”, and “&” for easier reading of the XML content.

Listing 1. Sample XML file format

```

1 <API>
2   <name>fare</name>
3   <description>
4     This API takes two parameters of age and distance
       traveled, and returns the fare based on age and
       distance rules.
5   </description>
6   <parameters>
7     <parameter>
8       <name>age</name><type>int</type>
9     </parameter>
10    <parameter>
11      <name>distance</name><type>int</type>
12    </parameter>
13  </parameters>
14  <business>
15    <case>
16      <condition>age < 4</condition>
17      <return>0</return>
18    </case>
19    <case>
20      <condition>age >= 4 && age <= 14</condition>
21      <subCondition>
22        <condition>distance >= 10</condition>
23        <return>130</return>
24      </subCondition>
25      <subCondition>
26        <condition>distance < 10</condition>
27        <return>100</return>
28      </subCondition>
29    </case>
30    <case>
31      <condition>age >= 15 && age <= 60</condition>
32      <subCondition>
33        <condition>distance <= 10</condition>
34        <return>160</return>
35      </subCondition>
36    </case>
37    <case>
38      <condition>age > 60</condition>
39      <subCondition>
40        <condition>distance < 10</condition>
41        <return>130</return>
42      </subCondition>
43    </case>
44    <case>
45      <condition>Others</condition>
46      <return>200</return>
47    </case>
48  </business>
49 </API>

```

In Listing 1, lines 2 to 5 show the general information of the API. Next, the list of parameters is shown in lines 6 to 13. The main business logic of the API

is shown in lines 14 to 48. In the `business` section, each case is included in the `case` tag with the `condition` and `subCondition` tags containing the business logic related to `age` and `distance` parameters. The `return` tag defines the returned value in each given case.

4 Automated Test Data Generation

4.1 Test Data Generation Using Boundary Value Analysis

This section presents the BVA method, which focuses on generating test data at the boundaries of the input domain. Currently, the framework supports generating BVA test data only for input parameters of primitive data types, including *short*, *int*, *long*, *float*, and *double*. Algorithm 1 details this process.

Algorithm 1: Boundary value test data generation

Input: L : List of pairs (p_i, V_i)
 p_i : an input parameter
 V_i : list of sorted values corresponding to p_i
 δ : a predefined value to generate test data at boundary values
Output: *BoundTD*: The generated test data set

```

1 begin
2   BoundTD  $\leftarrow \emptyset$ ; Norm  $\leftarrow \emptyset$ ;
3   foreach  $(p_i, V_i) \in L$  do
4     Normi  $\leftarrow \emptyset$ 
5     foreach  $v_t \in V_i$  do
6        $v_{norm} \leftarrow (v_t + v_{t+1})/2$ ; Normi  $\leftarrow$  Normi  $\cup$   $v_{norm}$ ;
7       Boundi.add( $v_{norm}$ ); Boundi.add( $v_t$ );
8       Boundi.add( $v_t - \delta$ ); Boundi.add( $v_t + \delta$ );
9     end
10    Norm.add( $(p_i, \text{Norm}_i)$ ); Bound.add( $(p_i, \text{Bound}_i)$ );
11  end
12  foreach  $(p_i, V_i) \in L$  do
13    Normi, Boundi  $\leftarrow$  Norm( $p_i$ ), Bound( $p_i$ );
14    Normtemp  $\leftarrow \{\text{Norm}(p_j) | j \neq i\}$ ;
15    foreach  $value \in \{\text{Bound}_i\} \cup \{\text{Norm}_i\}$  do
16       $td \leftarrow \{value\}$ ;
17      forall the  $norm \in \text{Cartesian\_Product}(\text{Norm}_{temp})$  do
18         $td \leftarrow td \cup \{norm\}$ ;
19      end
20      BoundTD.add( $td$ );
21    end
22  end
23  return BoundTD;
24 end

```

The algorithm takes a list L , which consists of pairs (p_i, V_i) . Here, p_i is an input parameter, and V_i is a list of its corresponding values retrieved from

the requirement XML file. The δ parameter is utilized to generate test data at the boundary values. The output of the algorithm is the set of generated BVA test data, denoted as *BoundTD*. In this algorithm, all boundary values and the norm values for each parameter p_i are stored in the *Bound* and *Norm* variables, respectively.

The algorithm starts by initializing the set of test data (*BoundTD*) as an empty set ($\emptyset$) and a list of norm values called *Norm* (line 2). For each pair (p_i, V_i) , the algorithm calculates the list of boundary value *Bound_i*, and *Norm_i* of p_i (lines 5–9). Then, the algorithm generates test data by the combination of each value of $\{Bound_i; Norm_i\}$ with one Cartesian product of all norm values of all other parameters (lines 12–22). The algorithm returns the generated boundary value test data stored in *BoundTD* set (line 23).

4.2 Test Data Generation Using Equivalence Class Partitioning

Equivalence Partitioning (ECP) [15,16] is a black-box testing technique that segments the input domain into various data classes, allowing for the generation of test data. Each ideal test data identifies errors that would typically require multiple arbitrary tests to detect. This approach evaluates equivalence classes based on specific input conditions. Upon receiving an input, the type of input condition is analyzed and categorized into equivalence classes. These classes describe the valid or invalid states for the given input conditions.

Algorithm 2: ECP test data generation

Input: L : List of pairs (p_i, V_i) , where
 p_i : an input parameter.
 V_i : the corresponding list of values retrieved from the XML requirement file.
Output: TD : The generated ECP test data set

```

1 begin
2    $TD \leftarrow \emptyset$ 
3   foreach  $(p_i, V_i) \in L$  do
4      $tempV_i \leftarrow \emptyset$ ;  $rep\_values \leftarrow \emptyset$ ;
5      $min \leftarrow Min(V_i)$ ;  $max \leftarrow Max(V_i)$ 
6      $eq\_classes \leftarrow eq\_classes \cup \{((-\infty, min), invalid); ((max, +\infty), invalid)\}$ ;
7      $eq\_classes \leftarrow eq\_classes \cup \{((min_i, max_i), valid) \mid 1 \leq i \leq n\}$ ;
8      $eq\_classes \leftarrow eq\_classes \cup \{((max_i, min_{i+1}), invalid) \mid 1 \leq i \leq n - 1\}$ ;
9     foreach  $k \in eq\_classes$  do
10       $rep\_value \leftarrow Random(k.Range.min, k.Range.max)$ ;
11       $tempV_i \leftarrow tempV_i \cup \{rep\_value\}$ ;
12    end
13  end
14   $TD \leftarrow$  Cartesian product of items in all  $tempV_i$ , where  $\forall 1 \leq i \leq n$ ;
15 end

```

This method is important in software testing to cover various scenarios efficiently without redundant testing. Normally, with an input parameter, random values that are either valid or invalid or within an equivalence range are chosen. In this research, we focus on primitive data types such as *short*, *int*, *long*, *float*, *double*, etc. The minimum and maximum values of the valid value domain are called *min* and *max*, respectively. The equivalence classes can include *valid* and *invalid* values. In the context of multiple ranges, valid ranges are denoted as (min_1, max_1) , (min_2, max_2) , ..., (min_n, max_n) , where $max_1 < min_2$, $max_2 < min_3$, ..., $max_{n-1} < min_n$. In this case, we define *min* as min_1 and *max* as max_n . The *invalid* values are values outside the allowable range which includes: values less than *min*; values greater than *max*; and values in the following ranges (max_1, min_2) , (max_2, min_3) , ..., (max_{n-1}, min_n) . In addition, the invalid value ranges include *invalid_below* (i.e., $invalid_below \equiv (-\infty, min)$) and *invalid_above* (i.e., $invalid_above \equiv (max, +\infty)$), respectively. Details of the ECP test data generation process are shown in Algorithm 2. Inputs of the algorithm is a list of pairs (*L*) where each item contains one input parameter (p_i) and the corresponding values generated from the XML requirement specification ($V_i = \{min_1; max_1; min_2; max_2; \dots; min_n; max_n\}$). The output of the algorithm is a set of ECP test data (*TD*). In this algorithm, the set *rep_value* stores all the representative values in each equivalence class of p_i .

Algorithm 2 begins by initializing an empty test data set named *TD* (line 2). For each pair of p_i and V_i in *L*, the algorithm determines *rep_values*, *min* as the lower boundary and *max* as the upper boundary (lines 4–5). Next, the algorithm defines the list of equivalence classes, each of which contains two properties: Range: the value range; and Type: *valid/invalid* (lines 6–8). We use the DOT notation to mention a property of an item in the list. The algorithm randomly generates one value for each equivalence class in *eq_classes* and adds it to *tempV_i* (lines 9–12). Finally, the algorithm calculates the Cartesian product of the items in all *tempV_i* for all parameters (p_i) of the unit (line 14) and returns.

5 Microservice API Test Execution

After we generate the test data set from Sect. 4 using the BVA and ECP methods, we must apply the unit requirements to specify the correct return value for the corresponding test data. This process results in test cases in a prescribed format (e.g., in a file).

Simultaneously, based on the unit requirements, the development team creates the corresponding API and deploys it to a predefined microservice within the system. Once the API is ready for testing, FAMRES calls the API to perform unit tests using the previously generated test cases with the REST Assured² libraries. Details of the API test execution process are shown in Algorithm 3. The algorithm takes in the microservice API server address (*ServerURL*), the name of the API under test (*APIName*), the name of the test case file (*TCFile*), and outputs the required test report (*Report*).

² <https://rest-assured.io/>.

Algorithm 3: Execute the API test cases

Input: *ServerURL*: The microservice server url
Input: *APIName*: The name of the API to test
Input: *TCFile*: The test case file
Output: *Report*: The test report

```

1 begin
2   Report  $\leftarrow \emptyset$ 
3   TCList  $\leftarrow \text{read\_all\_testcase}(\textit{TCFile})$ 
4   foreach testcase  $\in$  TCList do
5     result  $\leftarrow \text{Execute\_API}(\textit{ServerURL}, \textit{APIName}, \textit{testcase})$ 
6     Report  $\leftarrow \textit{Report} \cup \{\textit{result}\}$ 
7     Report  $\leftarrow \text{Caculate\_Aggregation\_Information}(\textit{result}, \textit{Report})$ 
8   end
9   return Report
10 end

```

The algorithm initializes the report as empty (line 2). It then reads all test cases from the test case file and stores them in *TCList* (line 3). Next, the algorithm calls the microservice API under test to retrieve the result (line 5). In this step, the algorithm uses services provided by the REST Assured libraries to call the API. Other supporting libraries can also be used for this task within the FAMRES framework. The result of the API execution is stored in *result*, which is then added to *Report*. The algorithm also adds other aggregation information to *Report*, such as the total number of successful or failed test cases (line 7). After processing all test cases, the algorithm returns *Report* (line 9).

6 Experiments

To evaluate the usefulness of the FAMRES framework, we implemented it in a tool of the same name³ using Java on IntelliJ IDE⁴ and conducted experiments with some common units⁵ in Java microservices. To demonstrate FAMRES's error-detection capability, we introduced some errors into the given units and then used the test data set generated by FAMRES to detect those errors. The experiments were performed on a machine with the following configuration: Apple M1 processor, 8GB RAM, and 256GB HDD.

We considered the following key indicators: the number of instrumented errors, the number of generated test data, the number of detected errors, and the time and memory required during the experiments. The experimental results are shown in Table 1. In this table, columns "Service name", "Err", "Test data", "Det. Err", "Time (ms)", and "Mem (MB)" show the name of the service under test, the number of instrumented errors, the number of test data generated by

³ <https://github.com/chichichin/AFrameworkforAutomatedTesting>.

⁴ <https://www.jetbrains.com/idea/>.

⁵ <https://github.com/chichichin/DataTest>.

the FAMRES framework, the number of detected errors, the time usage (in milliseconds), and the memory usage (in megabytes), respectively.

Table 1. Experimental results

No.	Service name	Err	Test data	Det. Err	Time (ms)	Mem (MB)
1	CalculateFare	4	76	3	2,463	31.00
2	GradeClassification	6	27	5	1,491	28.67
3	CalculateCustomerDiscount	8	75	7	2,263	14.00
4	DetermineExamResult	2	15	1	2,162	24.00
5	CalculateFinalPrice	5	30	4	2,013	31.49
6	CalculateLateBonus	4	109	3	3,021	24.83
7	MathEnglishGrade	3	81	3	3,048	11.66
8	CalculateShippingFare	8	57	6	2,398	41.51
9	CalculateTax	4	38	3	1,894	35.52
10	CalcualteShippingCost	3	40	2	2,217	34.50

From the experimental results, we have the following discussions:

- **Error detection capability:** FAMRES can detect all boundary values and equivalence class-related errors. However, FAMRES cannot detect other types of errors instrumented in the remaining services.
- **Number of generated test data:** The number of generated test data follows the boundary value analysis and equivalence class partitioning methods.
- **Time and memory usage:** The time and memory usage of FAMRES are quite small (i.e., 1,491–3,048 ms and 11–41.51 MB). This demonstrates the effectiveness of the FAMRES method, which can be applied in real projects.

7 Conclusions

We have introduced a fully automated framework named FAMRES for microservice API testing based on requirement specifications. The core concept of the framework is to propose an XML-based requirement specification structure and generate test data accordingly. The framework then uses the REST Assured libraries to invoke the microservice API with the generated test data and produces the necessary test report. The experimental results demonstrate the efficiency of its error detection capability.

Although the FAMRES framework has shown promising results, considerable work remains to implement it in real projects. First, we plan to conduct experiments with large-scale projects to evaluate FAMRES’s effectiveness. We also aim to incorporate additional test data generation methods from the given requirement specifications, such as decision table and pairwise testing. Moreover,

we will enhance the report to provide more useful information, such as code coverage and API invocation efficiency, and integrate the method into CI/CD or real-world DevOps pipelines for better use in a microservices development environment. Finally, integrating the requirement specification with the Object Constraint Language (OCL) [17] will enhance the framework's adaptability in practical applications.

References

1. Pahl, C., Jamshidi, P.: Microservices: a systematic mapping study. In: Proceedings of the 6th International Conference on Cloud Computing and Services Science, vol. 1 and 2, pp. 137–146 (2016)
2. Newman, S.: Building Microservices. O'Reilly Media, Inc. (2021)
3. Thomas, F.R.: Architectural styles and the design of network-based software architectures. Ph.D. dissertation, University of California, Irvine (2000)
4. Schirgi, T., Brenner, E.: Quality assurance for microservice architectures. In: 2021 IEEE 12th International Conference on Software Engineering and Service Science (ICSESS), pp. 76–80 (2021)
5. Bogner, J.: Industry practices and challenges for the evolvability assurance of microservices. *Empir. Softw. Eng.* **4**, 101–121 (2021)
6. Alluri, V.R.R., Katari, Thota, P.S., Reddy, S.G., Venkata, A.K.P.: Automated testing strategies for microservices: a DevOps approach. *Distrib. Learn. Broad Appl. Sci. Res.* **4**, 101–121 (2018)
7. Giamattei, L., Guerriero, A., Pietrantuono, R., Russo, S.: Automated grey-box testing of microservice architectures. In: 2022 IEEE 22nd International Conference on Software Quality, Reliability and Security (QRS), pp. 640–650 (2022)
8. Giamattei, L., Guerriero, A., Pietrantuono, R., Russo, S.: Automated functional and robustness testing of microservice architectures. *J. Syst. Softw.* **207**, 111857 (2024)
9. de Camargo, A., Salvadori, I., dos Santos Mello, R., Siqueira, F.: An architecture to automate performance tests on microservices. In: Proceedings of the 18th International Conference on Information Integration and Web-Based Applications and Services, pp. 422–429 (2016)
10. Waseem, M., Liang, P., Márquez, G., Di Salle, A.: Testing microservices architecture-based applications: a systematic mapping study. In: 2020 27th Asia-Pacific Software Engineering Conference (APSEC), pp. 119–128 (2020)
11. Gazzola, L., Goldstein, M., Mariani, L., Segall, I., Ussi, L.: Automatic ex-vivo regression testing of microservices. In: Proceedings of the IEEE/ACM 1st International Conference on Automation of Software Test, AST 2020, pp. 11–20. Association for Computing Machinery, New York (2020)
12. Cohen, D.M., Dalal, S.R., Fredman, M.L., Patton, G.C.: The AETG system: an approach to testing based on combinatorial design. *IEEE Trans. Softw. Eng.* **23**(7), 437–444 (1997)
13. Zhou, X., Yang, C., Gui, W.: State transition algorithm. *J. Industr. Manage. Optim.* **8**(4), 1039–1056 (2012)
14. Ferriday, C.: A review paper on decision table-based testing. Swansea University, CS339-2007, vol. 20, pp. 952–965 (2007)
15. Burnstein, I.: Practical Software Testing. Springer, Heidelberg (2003)

16. Aditya, P.M.: Foundations of Software Testing: Fundamental Algorithms and Techniques. Pearson, India (2007)
17. Cabot, J., Gogolla, M.: Object constraint language (OCL): a definitive guide. In: Bernardo, M., Cortellessa, V., Pierantonio, A. (eds.) SFM 2012. LNCS, vol. 7320, pp. 58–90. Springer, Heidelberg (2012). https://doi.org/10.1007/978-3-642-30982-3_3